

Adding Phidgets to a SPA

This guide assumes you've been through [1 DigitalInput.md](#) , [2 VoltageRatio.md](#) , and [3 VoltageOutput.md](#) , and have a single `.html` file that connects to a Phidget server and reads and writes to a few different channels.

Up to this point, everything has lived in that one HTML file. That's great for quick prototyping, but a fully featured web application is usually built with a framework instead of raw HTML and `<script>` tags.

The good news: nothing about Phidgets changes

Phidgets works the same way inside a front-end framework as it does in a plain HTML file — you still create a `NetworkConnection` , still open channels like `DigitalInput` or `VoltageRatioInput` , and still get callbacks like `onAttach` and `onStateChange` . The only real difference is *how* you get those values onto the screen. Instead of manually calling `document.getElementById(...)` and setting `.textContent` or `.classList` yourself, each framework has its own "reactivity" system — a `ref` in Vue, `useState` in React, a signal in Angular — that you update instead, and the framework takes care of reflecting that change in the UI.

What we're building

This repo contains examples of using Phidgets in popular front-end frameworks, each recreating the same button/slider/LED functionality from the earlier guides, but wired into that framework's own patterns instead of raw DOM calls.

If you're already familiar with one of these frameworks, jump straight to its example and see how the Phidget setup translates. If you haven't worked with any of them before, start with `4a_Vue` . It's what we use at Dimensional Innovations, and it's a fairly gentle framework to pick up as a beginner.

Trying it yourself

1. Open the `4a_Vue` folder and follow its README to install dependencies and start the dev server.
2. Compare `src/App.vue` against your finished `.html` file from `3_VoltageOutput.md` — look for where `NetworkConnection` and the channel classes are created, and how Vue's reactive state stands in for the `document.getElementById` calls.
3. Make sure a Phidget Network Server is running and reachable, with your sensors and output device plugged into the hub, and confirm the app behaves the same as your HTML version did.

Bonus Round

Once you're comfortable with the Vue example, try porting it (or your original HTML file) to a framework of your own choosing. The Phidget22 calls barely change — what you're really practicing is recognizing which parts of any framework's docs map to "update the UI" versus "talk to the hardware."